CampusEYE: AI-Based Classroom Photo Attendance Management System

Md. Rizvi Ahmed, Safayat Ibrahim, Rafiqul Islam, Md. Arebi Sarkar
Department of Electrical & Computer Engineering
North South University, Dhaka, Bangladesh
{rizvi.ahmed, safayat.ibrahim, rafiqul.islam, arebi.sarkar}@northsouth.edu

***Abstract*—This report presents the Week 1 implementation progress of the CampusEYE project, focusing on building the foundational system components. During this week, the project structure was established, including the database schema for student registration and attendance tracking, a real-time face registration module using DeepFace (FaceNet model), comprehensive project configuration settings, and an automated test suite for verification. The system uses OpenCV for face detection via Haar Cascade classifiers and DeepFace for generating 128-dimensional face embeddings. A SQLite database stores registered students with their averaged face embeddings and attendance records. The test suite validates database creation, student registration workflow, and data integrity across all components.**

***Index Terms*—Face Recognition, Attendance Management, DeepFace, FaceNet, OpenCV, Haar Cascade, SQLite, Student Registration, Face Embeddings**

I. Introduction

The CampusEYE project aims to develop an AI-powered classroom photo attendance management system that automates attendance tracking using facial recognition technology. The system is designed to address the limitations of traditional RFID-based systems, including card dependency, long queues during entry, and the prevalence of proxy attendance (proxy punching). By leveraging deep learning-based face recognition, the system can verify student identities without physical tokens, reducing both time and administrative overhead.

Week 1 focused on establishing the core infrastructure required for the system to function. This includes configuring project paths and settings, designing and implementing the SQLite database schema, developing a real-time student registration module using webcam input with DeepFace-based face embedding extraction, and creating a comprehensive test suite to validate all components. The architecture is designed to be modular, allowing seamless integration of additional features such as real-time recognition, dashboard reporting, and proxy detection in subsequent weeks.

II. Project Overview

A. Project Structure

The `attendance_system` directory was organized into a clean modular structure with separate packages for configuration, database management, utilities, and testing. The directory tree is shown below:

```
attendance_system/
  config.py
  requirements.txt
  database/
    db_manager.py
    attendance.db
  utils/
    registration.py
  tests/
    test_registration.py
  data/
    enrolled_students/
    unknown_faces/
    attendance_logs/
```

Listing 1. Project directory structure

B. Technology Stack

The system employs a Python-based technology stack as summarized in Table I.

TABLE I
Technology Stack

Component	Library/Tool
Face Detection	OpenCV (Haar Cascade)
Face Recognition	DeepFace (FaceNet)
Database	SQLite3
Numerical Operations	NumPy
Web Dashboard (future)	Streamlit
Data Logging	Pandas
Deep Learning Backend	TensorFlow

III. Implementation Details

A. Project Configuration (`config.py`)

The configuration module defines all system-wide constants and paths. It sets up directory structures for storing enrolled student data, unknown face captures, and attendance logs. All required directories are automatically created on module import. Key parameters are shown in Listing 2.

```
# config.py
FACE_ENCODING_DIM = 128          # FaceNet embedding
     dimension
RECOGNITION_THRESHOLD = 0.6      # Euclidean
    distance threshold
NUM_REGISTRATION_SAMPLES = 10    # Frames captured
    per student
CAMERA_INDEX = 0                 # Default webcam
FRAME_WIDTH, FRAME_HEIGHT = 640, 480
```

Listing 2. Key configuration constants

The recognition threshold of 0.6 determines how close a detected face embedding must be to a stored embedding to be

considered a match. This value will be tuned empirically in Week 2 based on real classroom data.

B. Database Layer (`database/db_manager.py`)

The database manager implements SQLite3-based persistence with two core tables. The `students` table stores registered student information including `student_id` (unique), `name`, and a face embedding serialized as a BLOB using Python's `pickle` module. The `attendance` table tracks daily attendance with `student_id`, `date`, `status`, and `timestamp`.

The schema enforces:

- `UNIQUE` constraint on `students.student_id` to prevent duplicate registrations.
- `UNIQUE(student_id, date)` on `attendance` to ensure one record per student per day.
- A foreign key from `attendance.student_id` to `students.student_id` for referential integrity.

The following functions were implemented:

```
init_database()      # Creates students & attendance
    tables
add_student()        # Inserts student with face
    embedding
get_all_students()   # Retrieves all registered
    students
get_student_count() # Returns total registered count
delete_student()     # Removes a student record
```

Listing 3. Database API functions

The embedding is converted to `np.float64` before serialization to ensure numerical precision is preserved across storage and retrieval cycles.

C. Student Registration (`utils/registration.py`)

The registration module provides real-time webcam-based student enrollment. It captures multiple face samples per student, detects faces using OpenCV's Haar Cascade classifier (eliminating the need for `dlib`), extracts face embeddings using DeepFace with the FaceNet model, and computes the average embedding across all samples for robustness.

The registration workflow is as follows:

1) Open webcam and start live video preview.
2) Detect faces in each frame using Haar Cascade.
3) Draw a bounding box around each detected face.
4) User presses `'C'` to capture a face sample.
5) Extract the 128-dimensional embedding via DeepFace/-FaceNet.
6) Repeat until `NUM_REGISTRATION_SAMPLES` are collected.
7) Compute the mean embedding across all collected samples.
8) Store the student record (ID, name, mean embedding) in the database.
9) User presses `'Q'` to quit the registration process.

The averaging of multiple samples mitigates the effects of variations in lighting, head pose, and facial expression during enrollment, resulting in a more robust reference embedding for future recognition.

D. Test Suite (`tests/test_registration.py`)

A comprehensive test suite validates all Week 1 components. The test functions cover database initialization and schema creation, interactive webcam-based student registration for 5 test students, and database verification checking embedding dimensions and data integrity.

The test students used for validation were:

- S001 – Alice Johnson
- S002 – Bob Smith
- S003 – Charlie Brown
- S004 – Diana Prince
- S005 – Eve Davis

Each test student is registered with 5 samples (instead of the default 10) for faster testing. The test suite verifies that:

- The database initializes without errors.
- All 5 students are successfully registered.
- Each stored embedding has exactly 128 dimensions.
- Each embedding is a valid `numpy.ndarray`.

E. Dependencies (`requirements.txt`)

The required Python packages for the system are listed in Listing 4. OpenCV provides face detection, DeepFace supplies the FaceNet embedding model, NumPy handles numerical operations, Pandas supports data manipulation, Streamlit is planned for the web dashboard, Pillow provides image handling, and TensorFlow serves as the deep learning backend for DeepFace.

```
opencv-python>=4.5.0
deepface>=0.0.79
numpy>=1.21.0
pandas>=1.3.0
streamlit>=1.10.0
Pillow>=8.0.0
tensorflow>=2.10.0
```

Listing 4. Python dependencies

IV. Task Reference (Proper Timeline)

The following tasks from the project timeline were completed during Week 1. Table II maps each task ID to its implementation.

TABLE II
Week 1 Task Completion

Task ID	Description
W1-T1	Project structure setup with config.py, paths, and directories
W1-T2	Requirements definition in requirements.txt
W1-T3	Database schema design (students & attendance tables)
W1-T4	Database initialization function (init_database)
W1-T5	Webcam capture loop with real-time face detection
W1-T6	Face embedding extraction using DeepFace/FaceNet
W1-T7	Multi-sample capture (10 frames averaged per student)
W1-T8	Database storage of student records with embeddings
W1-T9	Test suite and data verification

V. Conclusion & Next Steps

Week 1 successfully established the foundational infrastructure for the CampusEYE attendance system. The modular architecture separates concerns into configuration, database management, registration utilities, and testing. The use of DeepFace with FaceNet provides state-of-the-art face recognition capability without requiring the dlib library, simplifying deployment across different environments.

The following areas are planned for Week 2:

- Real-time face recognition using stored embeddings.
- Classroom photo capture and batch processing.
- Attendance marking logic with threshold-based matching.
- Reporting and dashboard visualization with Streamlit.
- Handling of unknown faces and attendance conflicts.

References

[1] S. I. Serengil and A. Ozpinar, "LightFace: A Hybrid Deep Face Recognition Framework," in *Proc. IEEE Conf. Innovations in Intelligent Systems and Applications (INISTA)*, 2020.

[2] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2015.

[3] P. Viola and M. Jones, "Rapid Object Detection using a Boosted Cascade of Simple Features," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2001.

[4] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[5] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2019.